

GitHub Documentation Guide

Dual-MCU IoT Sensor Node Project

[Directory Structure](#) · [File Content](#) · [Commit Discipline](#) · [Tags](#) · [README Evolution](#)

The Philosophy

GitHub is not a backup drive for your code. It is your **engineering record** — the artifact that proves you understood what you built. Every file, every commit, every tag in this project tells a story to anyone who reads it: a future employer, a collaborator, or you in 18 months when you've forgotten why you made a decision.

The distinction that matters: the **directory structure is the front-end** — what someone sees when they clone the repo. The **file content is the back-end** — what explains why it exists and how to use it. Both must be intentional.

The Directory Structure — Front-End

This is the full repo structure. Every directory and key file is explained below with its purpose and what must be in it.

```
dual-mcu-iot-node/
├── stm32/
│   ├── drivers/
│   │   ├── uart/
│   │   │   ├── uart.c
│   │   │   ├── uart.h
│   │   │   └── uart_test.c
│   │   ├── i2c/
│   │   │   ├── i2c.c
│   │   │   ├── i2c.h
│   │   │   └── i2c_test.c
│   │   ├── spi/
│   │   ├── gpio/
│   │   ├── timers/
│   │   ├── sensors/
│   │   ├── bmp280/
│   │   ├── dht22/
│   │   ├── hcsr04/
│   │   └── ssd1306/
│   ├── rtos/
│   └── FreeRTOS-Kernel/ ← git submodule
```

```
■ ■ ■■ freertos_config.h
■ ■ ■■ tasks/
■ ■ ■■ task_sensor.c
■ ■ ■■ task_uart_tx.c
■ ■ ■■ task_uart_rx.c
■ ■ ■■ task_cmd_handler.c
■ ■ ■■ task_health.c
■ ■■ bootloader/
■ ■ ■■ main.c
■ ■ ■■ flash.c
■ ■ ■■ flash.h
■ ■ ■■ bootloader.ld
■ ■ ■■ Makefile
■ ■■ app/
■ ■ ■■ main.c
■ ■ ■■ firmware_version.h
■ ■ ■■ application.ld
■ ■ ■■ Makefile
■ ■■ startup/
■ ■■ startup_stm32f411ceux.s
■ ■■ system_stm32f4xx.c
■■■ esp32/
■ ■■ main/
■ ■ ■■ app_main.c
■ ■ ■■ wifi_manager.c
■ ■ ■■ mqtt_client.c
■ ■ ■■ ota_handler.c
■ ■ ■■ protocol_esp.c
■ ■■ components/
■ ■■ CMakeLists.txt
■■■ protocol/
■ ■■ protocol_spec.md
■■■ pcb/
■ ■■ schematic/
■ ■■ layout/
■ ■■ gerbers/
■■■ docs/
■ ■■ architecture.md
■ ■■ build.md
■■■ scripts/
■ ■■ ota_push.py ← push firmware to MQTT from laptop
■■■ .gitignore
■■■ README.md
```

Directory Explanations — Back-End (The Why)

stm32/drivers/

One subdirectory per peripheral. This separation is deliberate: it forces each driver to be self-contained with its own header, implementation, and test file. When Phase 3 reuses the UART ring buffer code in the protocol engine, the clean boundary makes that dependency explicit rather than a tangle of included files.

stm32/drivers/[peripheral]/[driver]_test.c

The test harness for each driver. This file is compiled and flashed separately to verify the driver in isolation. It calls every function and prints PASS/FAIL over UART. It is NOT dead code — it is your regression suite. Every time you change a driver, you flash its test harness first.

stm32/drivers/sensors/

Sensor drivers are separated from peripheral drivers intentionally. A peripheral driver (UART, I2C) owns the hardware. A sensor driver (BMP280, DHT22) owns the application-layer interpretation of the data — compensation formulas, unit conversion, struct definitions. This layering means you can swap a sensor without touching the I2C driver.

stm32/rtos/FreeRTOS-Kernel/

This is a git submodule pointing to the official FreeRTOS/FreeRTOS-Kernel repo. Never copy FreeRTOS source files directly into your tree. A submodule means: (a) your repo records the exact kernel version you used, (b) upgrading means one command, (c) others who clone your repo can pull the exact same version. Set it up with: `git submodule add https://github.com/FreeRTOS/FreeRTOS-Kernel stm32/rtos/FreeRTOS-Kernel`

stm32/rtos/tasks/

Each FreeRTOS task in its own .c file. The task function, its queue/semaphore handles, and its stack size define are all in the same file. This makes task behaviour self-documenting — you don't have to search across files to understand what `task_sensor.c` does.

stm32/bootloader/

The bootloader is a completely separate binary with its own linker script and its own Makefile. It is never compiled together with the application. Keeping it in a separate directory prevents any accidental linking between the two. The bootloader Makefile produces `bootloader.bin`; the app Makefile produces `app.bin`.

stm32/app/firmware_version.h

A single header containing the firmware version string and integer. Example: `#define FW_VERSION_STR "1.2.0"` and `#define FW_VERSION_INT 0x010200`. The bootloader reads the integer from a known flash offset. The OTA pipeline uses it to verify a successful update. Every release bumps this manually.

esp32/

ESP-IDF project structure. The `esp32/` directory is a completely independent build system (CMake). It does not share source files with `stm32/`. The only shared artifact is the protocol specification in `protocol/` — both sides implement the same spec independently in their respective languages/environments.

protocol/protocol_spec.md

The single source of truth for the inter-MCU communication protocol. Both the STM32 and ESP32 implementations are derived from this document. It contains: frame format table, command byte definitions, byte stuffing rules, CRC algorithm and polynomial, timeout/retry parameters, and example byte sequences for each command. If STM32 and ESP32 disagree on protocol behaviour, this doc is the arbiter.

docs/architecture.md

The system-level design document. It describes the overall architecture — two MCUs, their roles, the dependency graph of subsystems, the flash memory layout, the FreeRTOS task diagram, and the OTA update flow. It is a living document updated at each phase boundary. It does NOT explain implementation details — those live in the code and driver headers.

docs/build.md

The operational document. A person who has never seen this project must be able to clone it, build it, and flash it by following build.md. It contains: toolchain install commands (Fedora-specific), build commands for STM32 and ESP32, flash commands with exact OpenOCD invocations, and hardware wiring for SWD. This is tested at every tagged release.

scripts/ota_push.py

A Python script that takes a firmware .bin file and pushes it to the MQTT broker in the correct sequence: OTA_START → chunks → OTA_END. This is how you trigger OTA updates from your laptop. It is part of the repo so the OTA workflow is reproducible by anyone.

.gitignore

Ignore: build/ and *.o and *.elf and *.bin and *.hex and *.map files from both stm32/ and esp32/ builds. Ignore esp32/build/. Never commit compiled artifacts — a clean repo with a working build.md is worth more than committing .hex files.

File Content Standards — What Goes in Each File

Driver Header File (uart.h)

Every driver header follows this structure. The discipline of documenting every function in the header pays off when you return to this code in month 6.

```
/**
 * @file uart.h
 * @brief UART1/USART2 register-level driver — STM32F411
 *
 * TX: polling and DMA.
 * RX: interrupt-driven ring buffer (UART_RX_BUF_SIZE bytes, power-of-2).
 * printf retargeting: _write() syscall overridden to call uart_write_buf().
 *
 * RM0383 reference sections used:
 * 30.3 USART functional description
 * 30.6.1 Status register (USART_SR)
 * 30.6.2 Data register (USART_DR)
 */

#pragma once
#include
#include

#define UART_RX_BUF_SIZE 256 /* Must be power of 2 */

void uart_init(uint32_t baud);
void uart_write_byte(uint8_t b);
void uart_write_buf(const uint8_t *buf, size_t len);
int uart_read_byte_timeout(uint8_t *out, uint32_t timeout_ms);
size_t uart_rx_available(void);
uint8_t uart_rx_get(void); /* Only call if available() > 0 */
```

Driver Implementation File (uart.c)

Every register write must be commented with its RM page number and field name. This is not optional housekeeping — it is the difference between code you wrote and code you can explain.

```
// RM0383 p.155 — RCC_APB2ENR bit 4: USART1EN
RCC->APB2ENR |= (1U << 4);

// RM0383 p.555 — USART_CR1: bit 13 UE (USART Enable)
// bit 3 TE (Transmitter Enable)
// bit 2 RE (Receiver Enable)
// bit 5 RXNEIE (RX not empty interrupt enable)
USART1->CR1 = (1U<<13) | (1U<<3) | (1U<<2) | (1U<<5);
```

Test File (uart_test.c)

```
/**
 * @file uart_test.c
 * @brief Standalone test harness for uart.c
 * Flash this instead of main app. Expected output on serial terminal:
 * [PASS] uart_write_byte
 * [PASS] uart_write_buf
 * [PASS] uart_rx ring buffer
 * [PASS] printf retargeting
 */
void uart_run_tests(void) {
    uart_init(115200);
    uart_write_buf((uint8_t*)" [TEST] UART driver\r\n", 20);
    printf("[PASS] printf retargeting\r\n");
    // ... ring buffer loopback test
}
```

Commit Discipline

Every commit in this repo answers: **what changed** and **why**. Not just what. The 'why' is what you cannot recover from the diff.

Commit Message Format:

```
type(scope): short summary (max 72 chars)
```

Optional body: one paragraph explaining WHY this change was made.

What problem did it solve? What did you try that didn't work?

Types: feat | fix | refactor | test | docs | chore

Scopes: uart | i2c | spi | freertos | protocol | bootloader | ota | esp32 | pcb

Real-World Examples (Good vs Bad):

✗ Bad commit message	✓ Good commit message
fixed bug	fix(i2c): add timeout to SR1 polling loop I2C was hanging indefinitely on bus lockup. STM32F4 I2C errata requires SR1 timeout to prevent permanent stall when slave holds SDA low. Added 1ms timeout with bus recovery fallback.
uart stuff	feat(uart): add interrupt-driven RX ring buffer Polling RX was blocking sensor acquisition task. ISR now fills 256-byte circular buffer. protocol_feed_byte() consumes from it non-blocking.
added freertos	feat(freertos): add FreeRTOS-Kernel as git submodule Adds portable/GCC/ARM_CM4F port. FreeRTOSConfig.h configured for 96MHz clock, 1ms tick, Heap_4. Stack overflow detection mode 2 enabled.

Tag Strategy

Tags are milestones, not versions of a shipped product. Each tag should be a point where the system is in a known-good, demonstrable state. Tag immediately after the phase exit gate is cleared.

Tag	Corresponds to	What it proves
v0.0-toolchain-verified	Phase 0 exit	Blink compiles, flashes via ST-Link, Makefile works
v0.1-peripheral-drivers	Phase 1 exit	All drivers tested: UART, I2C, SPI, Timers, all sensors respond
v0.2-freertos-tasks	Phase 2 exit (STM32)	FreeRTOS scheduler running, all tasks active, queues flowing
v0.2-esp32-mqtt	Phase 2 exit (ESP32)	WiFi connected, MQTT publishing to broker
v0.3-protocol	Phase 3 exit	Framed UART protocol exchanging data, CRC validation working

v0.4-bootloader	Phase 4 midpoint	Bootloader flashing and jumping to application
v0.4-ota-pipeline	Phase 4 exit	End-to-end OTA: laptop MQTT → ESP32 → STM32 flash → reboot
v1.0-integration	Phase 5 exit	72-hour soak passed. Watchdog enabled. System stable.
v1.1-pcb-bringup	Phase 6 exit	PCB fabricated, all firmware running on hardware

How to create an annotated tag:

```
git tag -a v0.1-peripheral-drivers -m "All peripheral drivers tested. \
UART ring buffer, I2C with timeout, SPI polling, Timer IC. \
All sensors responding: BMP280, DHT22, HC-SR04, SSD1306."
git push origin v0.1-peripheral-drivers
```


README Evolution

The README grows with the project. It is not written once and forgotten. At each tagged release, the README must accurately reflect the current state of the project. Here is the section structure and when each section appears.

Project Overview (Day 1)

One paragraph. What this is, what hardware it runs on, what it does. No marketing language. No bullet lists. One honest paragraph. Example: 'A dual-MCU IoT sensor node on STM32F411 (Black Pill) and ESP32 DevKit V1. Reads BMP280, DHT22, and HC-SR04 sensors over register-level I2C/SPI/1-wire drivers. Sends data via a custom framed UART protocol to ESP32, which publishes to MQTT. OTA firmware updates are delivered over WiFi via MQTT to a custom STM32 bootloader.'

Hardware Setup (Phase 0)

Wiring table: which STM32 pins connect to what. SWD connections for ST-Link. Power connections. Update this as you add sensors in Phase 1.

Build Instructions (Phase 0, kept current)

The exact commands to clone, build, and flash. Tested on Fedora. If someone follows these steps and it doesn't work, the README is wrong — fix it.

Architecture (Phase 2, updated through Phase 4)

Link to docs/architecture.md. A one-paragraph summary of the system design. Update the summary at each major milestone.

Firmware Versions (Phase 4 onward)

A table of version → what changed → GitHub tag. This becomes your changelog.

Project Journal (Throughout)

A short bulleted log at the bottom of the README, one entry per tag: date, tag name, one sentence on what was achieved and what was hard. This becomes the YouTube video description template.

Using GitHub Issues for Engineering Notes

When you discover a bug, a design question, or a 'must remember to do this' item — open a GitHub Issue immediately. Don't try to hold it in your head. Issues serve three purposes in this project:

- **Bug tracker:** When I2C locks up after 12 hours, open an issue: 'I2C bus lockup under sustained load'. Describe: what you observed, what you tried, what you think is happening. Close it with the commit that fixed it, linking to the commit hash. This is your debugging journal.
- **Design decisions:** Open an issue: 'Decision: CRC-16 vs CRC-32 for protocol'. Write both options and why you chose CRC-16. Close it. Now future-you knows why that decision was made.
- **Phase backlog:** Before starting Phase 3, open issues for each task: 'Implement byte stuffing', 'CRC-16 unit test on PC', 'Parser state machine'. Close them as you go. This is lightweight project management that costs 5 minutes.